Question 1

Q1. Write a Python program to demonstrate multiple inheritance.

1. Employee class has 3 data members EmployeeID, Gender (String), Salary and PerformanceRating(Out of 5) of type int. It has a get() function to get these details from the user.
2. JoiningDetail class has a data member DateOfJoining of type Date and a function getDoJ to get the Date of joining of employees.
3. Information Class uses the marks from Employee class and the DateOfJoining date from the JoiningDetail class to calculate the top 3 Employees based on their Ratings and then Display, using readData, all the details on these employees in Ascending order of their Date Of Joining.

Question 1: Multiple Inheritance

Write a Python program to demonstrate multiple inheritance.

1. Create an `Employee` class with the following data members:
   - `EmployeeID` (int)
   - `Gender` (string)
   - `Salary` (int)
   - `PerformanceRating` (int, out of 5)

   The class should include a method `get()` to obtain these details from the user.
2. Create a `JoiningDetail` class with:
   - a data member `DateOfJoining` (type: date)
   - a method `getDoJ()` to get the date of joining from the user.
3. Create an `Information` class that inherits from both `Employee` and `JoiningDetail`.
   - Use the `PerformanceRating` from `Employee` and the `DateOfJoining` from `JoiningDetail` to determine the top 3 employees based on their ratings.
   - Implement a `readData()` method to display all details of these top employees in **ascending order of their Date Of Joining**.

Answer 1

- My approach

Employee class

Employee class has 3 data members EmployeeID, Gender (String), Salary and PerformanceRating(Out of 5) of type int. It has a get() function to get these details from the user.

```
class Employee:
  def __init__(self, employee_id=None, gender=None, salary=None, performance_rating=None):
    self.employee_id = employee_id
    self.gender = gender
    self.salary = salary
    self.performance_rating = performance_rating

  def get(self, employee_id, gender, salary, performance_rating):
    self.employee_id = employee_id
    self.gender = gender
    self.salary = salary
    self.performance_rating = performance_rating

  def showEmployee(self):
    print(self.employee_id, self.gender, self.salary, self.performance_rating)
```

JoiningDetail class

JoiningDetail class has a data member DateOfJoining of type Date and a function getDoJ to get the Date of joining of employees.

[ ] ↳ 1 cell hidden

Information class

Information Class uses the marks from Employee class and the DateOfJoining date from the JoiningDetail class to calculate the top 3 Employees based on their Ratings and then Display, using readData, all the details on these employees in Ascending order of their Date Of Joining.

In accordance with the question's instructions, the Employee and JoiningDetail classes are required to collect data using the get() and getDoJ() methods. To respect this design, the Information class does not accept constructor arguments. Instead, it initializes all parent class variables to None via their default constructors and then populates them using the prescribed methods. This ensures compliance with the assignment's intent while maintaining clarity and separation of concerns between object creation and data input.

Since Information class is a child of Employee and JoiningDetail, one instance of Information class refers to only one Employee and JoiningDetail (of that particular employee). So, it can't store list of employees. I am using a class variable to store list of Employees and a class method to access that

```
from datetime import date

class Information(Employee, JoiningDetail):
  __employees = []

  def __init__(self):
    Employee.__init__(self)
    JoiningDetail.__init__(self)
    Information.__employees.append(self)

  def showInfo(self):
    print(f"Employee Id - {self.employee_id}", f"Gender - {self.gender}", f"Salary - {self.salary}", f"Performance Rating - {self.performanc

  @classmethod
  def readData(cls):
    top_3 = sorted(cls.__employees, key=lambda e: e.performance_rating, reverse=True)[:3]
    for i in sorted(top_3, key = lambda e : e.date_of_joining):
      i.showInfo()
      print("-"*30)
```

Instances and checking

- Order of Information instances with respect to seniority (ascending order of JoiningDate)
    1. information1
    2. information2
    3. information3
    4. information4
- Order of Information instances with respect to performance rating (descending order of PerformanceRating)
    1. information4
    2. information3
    3. information2
    4. information1

```
# More Senior employee (early joining date) with less rating
information1 = Information()
information1.get(11, "M", 18000, 3.0)
information1.getDoJ(date(2012, 10, 13))

# mid level Senior employee with average rating
information2 = Information()
information2.get(12, "F", 15000, 3.3)
information2.getDoJ(date(2016, 1, 20))

# mid level experienced employee with above average rating
information3 = Information()
information3.get(13, "M", 12000, 3.8)
information3.getDoJ(date(2018, 7, 16))

# More Junior employee (late joining date) with high rating
```

```
information4 = Information()
information4.get(14, "F", 10000, 4.1)
information4.getDoJ(date(2021, 2, 9))
```

Top 3 high rated employees (information2, information3, information2) are selected. But ordered according to seniority (ascending order of the JoiningDate)

```
Information.readData()
```

```
Employee Id - 12
Gender - F
Salary - 15000
Performance Rating - 3.3
Date of Joining - 2016-01-20
------------------------------
Employee Id - 13
Gender - M
Salary - 12000
Performance Rating - 3.8
Date of Joining - 2018-07-16
------------------------------
Employee Id - 14
Gender - F
Salary - 10000
Performance Rating - 4.1
Date of Joining - 2021-02-09
------------------------------
```

```
# Extra - Checking with data mangling
for i in information1._Information__employees:
  i.showEmployee()
  i.showDoJ()
```

```
11 M 18000 3.0
2012-10-13
12 F 15000 3.3
2016-01-20
13 M 12000 3.8
2018-07-16
14 F 10000 4.1
2021-02-09
```

Answer 2

- the trainer's approach in one on one session

```
from datetime import datetime

class Employee:
  def __init__(self):
    self.EmployeeID=""
    self.Gender=""
    self.Salary=0
    self.PerformanceRating=0

  def get(self):
    self.EmployeeID=input("Enter Employee ID: ")
    self.Gender=input("Enter Gender: ")
    self.Salary=int(input("Enter Salary: "))
    self.PerformanceRating=int(input("Enter Performance Rating(out of 5): "))


class JoiningDetail:
  def __init__(self):
    self.DateOfJoining=""

  def getDoJ(self):
    date=input("Enter Date of Joining (YYYY-MM-DD): ")
    self.DateOfJoining=datetime.strptime(date,"%Y-%m-%d")


class Information(Employee,JoiningDetail):
  def __init__(self):
    Employee.__init__(self)
    JoiningDetail.__init__(self)
```

```
  def readData(self):
    print("Employee ID: ",self.EmployeeID)
    print(f"Gender: {self.Gender}")
    print(f"Salary: {self.Salary}")
    print(f"Performance Rating(OUT OF 5): {self.PerformanceRating}")
    print(f"Date of Joining: {self.DateOfJoining.date()}") #


employees=[]

n=int(input("Enter number of employees: "))
for _ in range(n):
  info=Information()
  info.get()
  info.getDoJ()
  employees.append(info)


top_employees=sorted(employees, key=lambda x:x.PerformanceRating, reverse=True)[:3]

TDoJ=sorted(top_employees, key=lambda x:x.DateOfJoining)

print("Top 3 Employees based on Performance: (sorted by Date of Joining): ")
for emp in TDoJ:
  print("-"*30)
  emp.readData()
```

```
Enter number of employees: 2
Enter Employee ID: AB!
Enter Gender: Female
Enter Salary: 100000
Enter Performance Rating(out of 5): 4
Enter Date of Joining (YYYY-MM-DD): 2024-06-10
Enter Employee ID: AB2
Enter Gender: Male
Enter Salary: 200000
Enter Performance Rating(out of 5): 5
Enter Date of Joining (YYYY-MM-DD): 2020-02-09
Top 3 Employees based on Performance: (sorted by Date of Joining):
------------------------------
Employee ID:  AB2
Gender: Male
Salary: 200000
Performance Rating(OUT OF 5): 5
Date of Joining: 2020-02-09
------------------------------
Employee ID:  AB!
Gender: Female
Salary: 100000
Performance Rating(OUT OF 5): 4
Date of Joining: 2024-06-10
```

Question 2

Q.2 Write a Python program to demonstrate Polymorphism.

1. Class **Vehicle** with a parameterized function **Fare**, that takes input value as fare and returns it to calling Objects.
2. Create five separate variables **Bus, Car, Train, Truck and Ship** that call the **Fare** function.
3. Use a third variable TotalFare to store the sum of fare for each Vehicle Type.
4. Print the **TotalFare**.

Answer

The question is unclear, so I am explorintg various ways to answer

Answer 1

Most simple and straight forward answer

```
class Vehicle:
    def Fare(self, fare):
        """
        A parameterized function to return the fare for a vehicle.

        Args:
            fare (int or float): The fare amount.

        Returns:
            int or float: The fare amount.
        """
        return fare

# Creating instances of the Vehicle class
bus = Vehicle()
car = Vehicle()
train = Vehicle()
truck = Vehicle()
ship = Vehicle()

# Calling the Fare function for each vehicle and storing the fare
bus_fare = bus.Fare(50)
car_fare = car.Fare(100)
train_fare = train.Fare(200)
truck_fare = truck.Fare(150)
ship_fare = ship.Fare(300)

# Calculating the total fare
TotalFare1 = bus_fare + car_fare + train_fare + truck_fare + ship_fare

# Printing the total fare
print(f"Total Fare (individually added): {TotalFare1}")

# Calculating using loop - somewhat justifies Polymorphism
vehicles = [bus, car, train, truck , ship]

TotalFare = 0
for vehicle in vehicles:
  TotalFare += vehicle.Fare(100)

print(f"TotalFare through loop, {100} each - {TotalFare}")
```

```
Total Fare (individually added): 800
TotalFare through loop, 100 each - 500
```

Answer 2

- Basic Polymorphism implementation using Inheritance
- add ons - constructor
- helper function average_fare
- Child classes have their own minimuml, maximum and average fares
- Instace creation uses the average fare as Fare

```
class Vehicle:
  def __init__(self):
    self.fare = 0

  def average_fare(self):
    return self.fare

  def Fare(self, fare):
    self.fare = fare
    return self.fare

class Bus(Vehicle):
  def __init__(self):
    super().__init__()
    self.minimum_fare = 10
    self.maximum_fare = 50

  def average_fare(self):
    return (self.minimum_fare + self.maximum_fare) // 2

  def Fare(self, fare):
```

```
        if self.minimum_fare <= fare <= self.maximum_fare:
          return super().Fare(fare)
        return f"Bus Fare be between {self.minimum_fare} and {self.maximum_fare} - both inclusive"

    class Car(Vehicle):
      def __init__(self):
        super().__init__()
        self.minimum_fare = 150
        self.maximum_fare = 5000

      def average_fare(self):
        return (self.minimum_fare + self.maximum_fare) // 2

      def Fare(self, fare):
        if self.minimum_fare <= fare <= self.maximum_fare:
          return super().Fare(fare)
        return f"Car Fare be between {self.minimum_fare} and {self.maximum_fare} - both inclusive"

    class Train(Vehicle):
      def __init__(self):
        super().__init__()
        self.minimum_fare = 100
        self.maximum_fare = 3000

      def average_fare(self):
        return (self.minimum_fare + self.maximum_fare) // 2

      def Fare(self, fare):
        if self.minimum_fare <= fare <= self.maximum_fare:
          return super().Fare(fare)
        return f"Train Fare be between {self.minimum_fare} and {self.maximum_fare} - both inclusive"

    class Truck(Vehicle):
      def __init__(self):
        super().__init__()
        self.minimum_fare = 1000
        self.maximum_fare = 20000

      def average_fare(self):
        return (self.minimum_fare + self.maximum_fare) // 2

      def Fare(self, fare):
        if self.minimum_fare <= fare <= self.maximum_fare:
          return super().Fare(fare)
        return f"Truck Fare be between {self.minimum_fare} and {self.maximum_fare} - both inclusive"

    class Ship(Vehicle):
      def __init__(self):
        super().__init__()
        self.minimum_fare = 3000
        self.maximum_fare = 100000

      def average_fare(self):
        return (self.minimum_fare + self.maximum_fare) // 2

      def Fare(self, fare):
        if self.minimum_fare <= fare <= self.maximum_fare:
          return super().Fare(fare)
        return f"Ship Fare be between {self.minimum_fare} and {self.maximum_fare} - both inclusive"


    vehicles = [Bus(), Car(), Train(), Truck(), Ship()]

    TotalFare = 0
    for vehicle in vehicles:
      TotalFare += vehicle.Fare(vehicle.average_fare())

    print(TotalFare)
```

```
66155
```

Answer 3

- More detailed usage of Polymorphism
- Below approach is inspired from the Reddit comment [Reference Link]

- In this approach each subclass when called adds up to the cost of particualar mode of transport separately

```
class Vehicle():  # base class
    TotalFare = {}

    def fare(self, amt: int) -> int:
        return amt


class Bus(Vehicle):  # Child class Inherit from Vehicle and fare method overriden.
    def __init__(self):
        if "Bus" not in self.TotalFare:
            self.TotalFare["Bus"] = 0

    def fare(self, amt: int) -> int:
        self.TotalFare["Bus"] += amt
        return amt

class Car(Vehicle):  # Child class Inherit from Vehicle and fare method overriden.
    def __init__(self):
        if "Car" not in self.TotalFare:
            self.TotalFare["Car"] = 0

    def fare(self, amt: int) -> int:
        self.TotalFare["Car"] += amt
        return amt


class Train(Vehicle):  # Child class Inherit from Vehicle and fare method overriden.
    def __init__(self):
        if "Train" not in self.TotalFare:
            self.TotalFare["Train"] = 0

    def fare(self, amt: int) -> int:
        self.TotalFare["Train"] += amt
        return amt


class Truck(Vehicle):  # Child class Inherit from Vehicle and fare method overriden.
    def __init__(self):
        if "Truck" not in self.TotalFare:
            self.TotalFare["Truck"] = 0

    def fare(self, amt: int) -> int:
        self.TotalFare["Truck"] += amt
        return amt


class Ship(Vehicle):  # Child class Inherit from Vehicle and fare method overriden.
    def __init__(self):
        if "Ship" not in self.TotalFare:
            self.TotalFare["Ship"] = 0

    def fare(self, amt: int) -> int:
        self.TotalFare["Ship"] += amt
        return amt


# class instances.
veh = Vehicle()
bus = Bus()
car = Car()
train = Train()
truck = Truck()
ship = Ship()


# Short story where fare is called from each subclass.

print(f"Today, took an Uber to help a friend move. It cost Rs.{car.fare(400)}, mostly because I had to ride a ferry.")
print(f"That added another Rs.{ship.fare(2000)}, but was the quickest route during rush hour.")
print(f"When I arrived, I realized my friend hadn't rented a truck. I took the Uber to the local rental shop.")
print(f"The truck ran me Rs.{truck.fare(7000)} for the day. Paul will pay me back later. After we loaded, we opted to take the toll tunnel,
print(f"Several tolls later Rs.{truck.fare(2000)}, we arrived at the new location and unloaded.")
print(f"We dropped the truck off at the rental location near his new house. Walking out the door, Paul realized he forgot his lucky jacket
print(f"We hop on the bus (Rs.{bus.fare(40)}) and rode to the local train station and caught a ride back to his old neighborhood. Rs.{train
```

```
print(f"I start complaining to Paul about the money and time we're spending for this jacket as I approve another Rs.{car.fare(600)} for the
print(f"Paul grabs his jacket and does one last check. The Uber driver decided to wait for the return trip. Another Rs.{car.fare(300)}, plus
print(f"Another Rs.{train.fare(100)} train ride and a claustrophobic Rs.{bus.fare(40)} bus trip back to his neighborhood.")
print("We have some delivered sushi and declare victory on the day, then I start looking for a ride home.")
print("I briefly consider just buying a car as I wait for my new Uber driver to arrive.")
print(f"Rs.{car.fare(400)} later, and I'm safely in bed.")
print(f"I decide to run through my expenses for the day:")


# Printing TotalFare
total = 0
for key, cost in veh.TotalFare.items():
    print(f"{key}: Rs.{cost}")
    total += cost
print(f"\nTotal: Rs.{total}")
```

```
Today, took an Uber to help a friend move. It cost Rs.400, mostly because I had to ride a ferry.
That added another Rs.2000, but was the quickest route during rush hour.
When I arrived, I realized my friend hadn't rented a truck. I took the Uber to the local rental shop.
The truck ran me Rs.7000 for the day. Paul will pay me back later. After we loaded, we opted to take the toll tunnel, which added anothe
Several tolls later Rs.2000, we arrived at the new location and unloaded.
We dropped the truck off at the rental location near his new house. Walking out the door, Paul realized he forgot his lucky jacket on t
We hop on the bus (Rs.40) and rode to the local train station and caught a ride back to his old neighborhood. Rs.100 later, and we're c
I start complaining to Paul about the money and time we're spending for this jacket as I approve another Rs.600 for the car ride.
Paul grabs his jacket and does one last check. The Uber driver decided to wait for the return trip. Another Rs.300, plus an extra Rs.10
Another Rs.100 train ride and a claustrophobic Rs.40 bus trip back to his neighborhood.
We have some delivered sushi and declare victory on the day, then I start looking for a ride home.
I briefly consider just buying a car as I wait for my new Uber driver to arrive.
Rs.400 later, and I'm safely in bed.
I decide to run through my expenses for the day:
Bus: Rs.80
Car: Rs.1800
Train: Rs.200
Truck: Rs.9500
Ship: Rs.2000

Total: Rs.13580
```

Question 3

Q3. Consider an ongoing test cricket series. Following are the names of the players and their scores in the test1 and 2.

Test Match 1 :

Dhoni : 56 , Balaji : 94

Test Match 2 :

Balaji : 80 , Dravid : 105

Calculate the highest number of runs scored by an individual cricketer in both of the matches. Create a python function Max_Score (M) that reads a dictionary M that recognizes the player with the highest total score. This function will return (Top player , Total Score) . You can consider the Top player as String who is the highest scorer and Top score as Integer .

Input : Max_Score({‘test1’:{‘Dhoni’:56, ‘Balaji : 85}, ‘test2’:{‘Dhoni’ 87, ‘Balaji”:200}})

Output : (‘Balaji ‘ , 200)

The question is unclear, so trying different approaches

Answer 1

- Basic straight forward implementation
- Given the input is hardcoded, the below code works for only the dict with 2 tests - `test1` and `test2`
- The keys are taken in set to avoid repetition of keys when combining
- key for max function is 2nd element (`x[1]`) of the tuple which is the total score of the player

```
def Max_Score(M):
  total_score = {k: M.get('test1').get(k, 0) + M.get('test2').get(k, 0)
          for k in set(M.get('test1')).union(set(M.get('test2')))}
  return max(total_score.items(), key= lambda x: x[1])
```

```
Max_Score({'test1':{'Dhoni':56, 'Balaji' : 85, '1' : 100}, 'test2':{'Dhoni' : 87, 'Balaji':200, '2' : 200}})
```

```
('Balaji', 285)
```

Answer 2: Using `Counter`

- `Counter` is a dictionary subclass from Python's `collections` module, which helps in counting hashable items.
- Here, it is used to **merge two dictionaries** (`test1` and `test2`) by **automatically summing values** for matching keys (i.e., player names).
- Counter is dictionary which the keys as elements and the values as the count or frequency of particular element.
- It is used to count the occurrences of elements in list
- Consider in the match each time Dhoni takes a run, his name is added to the list. Same to Balaji.
- The list would have been like [Dhoni, Dhoni, Balaji, Dhoni, ...]
- Now for the match score would be count of Dhoni and Balaji in form of dictionary which is already given to us as input.
- The `most_common()` function is used to sort the elements decending order of the count (here count is the score)
- Reference links -
    1. Counter in Python - Geeks for Geeks
    2. Sorting in Counter - Stack overflow
    3. Counter - most_common documentation

```
from collections import Counter

def Max_Score_Using_Counter(M):
  total_score = Counter(M['test1']) + Counter(M['test2'])
  return total_score.most_common(1)[0]


Max_Score_Using_Counter({'test1':{'Dhoni':56, 'Balaji' : 85, '1' : 100}, 'test2':{'Dhoni' : 87, 'Balaji':200, '2' : 200}})
```

```
('Balaji', 285)
```

Answer 3

- Extending the implementation to handle N tests
- Mannual loop method

```
def Max_Score_for_N_tests(M):
  players = {player for test_match_scores in M.values() for player in test_match_scores}
  total_score = {}
  for test_match_scores in M.values():
    for player, score in test_match_scores.items():
      total_score[player] = total_score.get(player, 0) + score
  return max(total_score.items(), key= lambda x: x[1])

Max_Score_for_N_tests({'test1':{'Dhoni':56, 'Balaji' : 85, '1' : 100}, 'test2':{'Dhoni' : 87, 'Balaji':200, '2' : 200}})
```

```
('Balaji', 285)
```

Answer 4

- Extending the implementation to handle N tests
- Using `Counter`

```
from collections import Counter

def Max_Score_for_N_tests_using_counter(M):
  total_score = sum((Counter(d) for d in M.values()), Counter())
  return total_score.most_common(1)[0]


Max_Score_for_N_tests_using_counter({'test1':{'Dhoni':56, 'Balaji' : 85, '1' : 100}, 'test2':{'Dhoni' : 87, 'Balaji':200, '2' : 200}})
```

```
('Balaji', 285)
```

Question 4

Q4. Create a simple Card game in which there are 8 cards which are randomly chosen from a deck. The first card is shown face up. The game asks the player to predict whether the next card in the selection will have a higher or lower value than the currently showing card.

For example, say the card that's shown is a 3. The player chooses "higher," and the next card is shown. If that card has a higher value, the player is correct. In this example, if the player had chosen "lower," they would have been incorrect. If the player guesses correctly, they get 20 points. If they choose incorrectly, they lose 15 points. If the next card to be turned over has the same value as the previous card, the player is incorrect.

Deck building

- suits is list of 4 suits in standard deck
- ranks is [A, 2, 3, 4, 5, 6, 7, 8, 9, 10, J, Q, K] list - 13 cards in each suit from lower rank to higher rank
- deck is list of tuples each represent a card, 2 elemets in each tuple 1st is suit, 2nd is rank. This is built using list comprehension

```
suits = ['♠', '♥', '♦', '♣']
ranks = ['A'] + [str(n) for n in range(2, 11)] + ['J', 'Q', 'K']
deck = [(suit, rank) for suit in suits for rank in ranks]
print(deck)
```

```
[('♠', 'A'), ('♠', '2'), ('♠', '3'), ('♠', '4'), ('♠', '5'), ('♠', '6'), ('♠', '7'), ('♠', '8'), ('♠', '9'), ('♠', '10'), ('♠', 'J'), (
```

```
len(deck)
```

```
52
```

Card selection

- Hand is the randomly selected cards kept in hand

```
import random
hand = random.sample(deck, 8)
```

- rank is the function to get the rank of card in number from 1 to 13.
- A is 1
- 2 to 10 as it is
- J is 11
- Q is 12
- K is 13

```
rank = lambda card : 1 if card[1] == 'A' else 11 if card[1] == 'J' else 12 if card[1] == 'Q' else 13 if card[1] == 'K' else int(card[1])
```

Game Starts !!

- only 2 options - higher or lower can be selected, no option for guessing equal.
- equal rank in next card is unfortunately termed as loss irrespective of the guess
- Choice is case sencitive. Invalid choice is not counted and the chance for that card is given again

```
points = 0
i = 1
while(i < len(hand)):
  print(f"Faced up card - {i}th card - {hand[i-1][0]}{hand[i-1][1]}")
  choice = input("Choose whether next card is higher or lower")
  if(choice not in ["higher", "lower"]):
    print("Invalid choice, try again")
    continue
  print(f"revealing {i+1}th card - {hand[i][0]}{hand[i][1]}")
  points += 20 if choice=="higher" and rank(hand[i])>rank(hand[i-1]) or choice=="lower" and rank(hand[i])<rank(hand[i-1]) else -15
  print(f"points - {points}")
  i+=1
```

```
Faced up card - 1th card - ♠Q
Choose whether next card is higher or lowerlower
revealing 2th card - ♠7
points - 20
```

```
Faced up card - 2th card - ♠7
Choose whether next card is higher or lowerlower
revealing 3th card - ♦5
points - 40
Faced up card - 3th card - ♦5
Choose whether next card is higher or lowerhigher
revealing 4th card - ♦8
points - 60
Faced up card - 4th card - ♦8
Choose whether next card is higher or lowerlower
revealing 5th card - ♣Q
points - 45
Faced up card - 5th card - ♣Q
Choose whether next card is higher or lowerhigher
revealing 6th card - ♣3
points - 30
Faced up card - 6th card - ♣3
Choose whether next card is higher or lowerlower
revealing 7th card - ♦6
points - 15
Faced up card - 7th card - ♦6
Choose whether next card is higher or lowerlower
revealing 8th card - ♠10
points - 0
```

- Fun Fact -

> I have tried 2 times, both the times I guessed everything right but this time I got a perfect 0 as score

Question 5

Q5. Create an empty dictionary called Car_0 . Then fill the dictionary with Keys : color , speed , X_position and Y_position.

car_0 = {'x_position': 10, 'y_position': 72, 'speed': 'medium'} .

a) If the speed is slow the coordinates of the X_pos get incremented by 2.

b) If the speed is Medium the coordinates of the X_pos gets incremented by 9

c) Now if the speed is Fast the coordinates of the X_pos gets incremented by 22.

Print the modified dictionary.

```
Car_0 = {}


print(Car_0)
```

```
{}
```

```
# We need to fill 4 key value pairs, fill 2 using basic key value assignment, other 2 using update
Car_0["color"] = "blue"
Car_0["speed"] = 'medium'


Car_0.update({"X_position" : 10, "Y_position" : 72})


# Speed input from user is case insensitive
speed = input("Enter the speed : slow, medium or fast \n").lower()
```

```
Enter the speed : slow, medium or fast
Medium
```

```
if(speed in ['slow', 'medium', 'fast']):
  Car_0['speed'] = speed
  Car_0['X_position'] += 2 if speed=='slow' else 9 if speed=='medium' else 22
else:
  print("invalid input")


Car_0
```

```
{'color': 'blue', 'speed': 'medium', 'X_position': 19, 'Y_position': 72}
```

Question 6

Q6. Show a basic implementation of abstraction in python using the abstract classes.

1. Create an abstract class in python.
2. Implement abstraction with the other classes and base class as abstract class.

Abstraction in Python

Abstraction is the process of hiding implementation details and exposing only the required functionalities. In Python, it is implemented using abstract base classes (`ABC`) and `@abstractmethod`.

What This Code Demonstrates

- An abstract class `Account` defines common structure for all account types.
- It includes:
  - An abstract method `acc_type()` with base logic.
  - An abstract method `minimum_balance()` with no logic (just a contract).
  - A concrete method `currency_used()` reused by subclasses.
- Subclasses `SavingsAccount` and `CurrentAccount` implement the abstract methods differently, enforcing abstraction and showing polymorphism.

```
from abc import ABC, abstractmethod

class Account(ABC):
  def __init__(self, bank_name):
    self.bank_name = bank_name

  @abstractmethod
  def acc_type(self):
    return "Bank Account of " + self.bank_name

  @abstractmethod
  def minimum_balance(self):
    pass

  def currency_used(self):
    return "Rs."

class SavingsAccount(Account):
  def __init__(self, bank_name, account_number):
   super().__init__(bank_name)
   self.account_number = account_number

  def acc_type(self):
    return "Savings " + super().acc_type()

  def minimum_balance(self):
    return super().currency_used() + "1000"

class CurrentAccount(Account):
  def __init__(self, bank_name, account_number):
   super().__init__(bank_name)
   self.account_number = account_number

  def acc_type(self):
    return "Current " + super().acc_type()

  def minimum_balance(self):
    return super().currency_used() + "5000"
```

```
Start coding or generate with AI.
```

```
savings_account = SavingsAccount("ABC Bank", "1234")
```

```
savings_account.acc_type()
```

```
'Savings Bank Account of ABC Bank'
```

```
savings_account.minimum_balance()
```

```
'Rs 1000'
```

```
current_account = CurrentAccount("ABC Bank", "5678")
```

```
current_account.acc_type()
```

```
'Current Bank Account of ABC Bank'
```

```
current_account.minimum_balance()
```

```
'Rs 5000'
```

Question 7

Q7. Create a program in python to demonstrate Polymorphism.

1. Make use of private and protected members using python name mangling techniques.

```
class Vehicle:
  def __init__(self, color, max_speed, num_of_wheels, registration_number, insurance_expiry_date):
    self.color = color
    self._max_speed = max_speed
    self._num_of_wheels = num_of_wheels
    self.__registration_number = registration_number
    self.__insurance_expiry_date = insurance_expiry_date

  def get_registration_number(self):
    return self.__registration_number

  def update_insurance(self, insurance_expiry_date):
    self.__insurance_expiry_date = insurance_expiry_date

  def get_insurance_expiry_date(self):
    return self.__insurance_expiry_date

  def start_vehicle(self):
    pass

  def speed_limit_alert(self):
    pass

  def show(self):
    print(f"Registration Number : {self.__registration_number}")
    print(f"Color : {self.color}")
    print(f"Maximum Speed in km/hr : {self._max_speed}")
    print(f"Number of Wheels : {self._num_of_wheels}")
    print(f"Insurance Expiry date in YYYY-MM-DD : {self.__insurance_expiry_date}")

class Bike(Vehicle):
  def __init__(self, color, max_speed, registration_number, insurance_expiry_date, has_kickstart):
    super().__init__(color, max_speed, 2, registration_number, insurance_expiry_date)
    self.has_kickstart = has_kickstart

  def start_vehicle(self):
    print(f"The {self._num_of_wheels} wheels set on adventure!!! Vroom")

  def speed_limit_alert(self):
    print(f"Bike max speed is {self._max_speed} km/h — ride safe!")

  def show(self):
    super().show()
    print(f"The bike has {'no ' if not self.has_kickstart else ''}kick start")

class Car(Vehicle):
  def __init__(self, color, max_speed, registration_number, insurance_expiry_date, has_sunroof):
    super().__init__(color, max_speed, 4, registration_number, insurance_expiry_date)
    self.has_sunroof = has_sunroof

  def start_vehicle(self):
    print(f"Starting the {self._num_of_wheels} wheeler!!! Engine-on")
```

```
  def speed_limit_alert(self):
    print(f"car max speed is {self._max_speed} km/h — ride safe!")

  def show(self):
    super().show()
    print(f"The Car has {'no ' if not self.has_sunroof else ''}sun roof")


from datetime import date
# as we created Bike and Car as subclasses. Now we create object for base class

# a three wheeler auto
auto1 = Vehicle('Yellow', 60, 3, 'TN36 AA 1111', date(2026, 12, 31))

auto1.get_registration_number()
print(auto1.get_insurance_expiry_date())
auto1.update_insurance(date(2027, 12, 31))
auto1.show()
```

```
2026-12-31
Registration Number : TN36 AA 1111
Color : Yellow
Maximum Speed in km/hr : 60
Number of Wheels : 3
Insurance Expiry date in YYYY-MM-DD : 2027-12-31
```

```
bike1 = Bike('Blue', 120, 'TN36 BB 1111', date(2027, 12, 31), True)

bike1.get_registration_number()
print(bike1.get_insurance_expiry_date())
bike1.update_insurance(date(2028, 12, 31))
bike1.start_vehicle()
bike1.speed_limit_alert()
bike1.show()
```

```
2027-12-31
The 2 wheels set on adventure!!! Vroom
Bike max speed is 120 km/h — ride safe!
Registration Number : TN36 BB 1111
Color : Blue
Maximum Speed in km/hr : 120
Number of Wheels : 2
Insurance Expiry date in YYYY-MM-DD : 2028-12-31
The bike has kick start
```

```
bike2 = Bike('Red', 130, 'TN36 BY 1111', date(2027, 11, 30), False)

bike2.get_registration_number()
print(bike2.get_insurance_expiry_date())
bike2.update_insurance(date(2028, 11, 30))
bike2.start_vehicle()
bike2.speed_limit_alert()
bike2.show()
```

```
2027-11-30
The 2 wheels set on adventure!!! Vroom
Bike max speed is 130 km/h — ride safe!
Registration Number : TN36 BY 1111
Color : Red
Maximum Speed in km/hr : 130
Number of Wheels : 2
Insurance Expiry date in YYYY-MM-DD : 2028-11-30
The bike has no kick start
```

```
car1 = Car('Black', 160, 'TN36 CC 1111', date(2028, 6, 30), True)

car1.get_registration_number()
print(car1.get_insurance_expiry_date())
car1.update_insurance(date(2029, 6, 30))
car1.start_vehicle()
car1.speed_limit_alert()
car1.show()
```

```
2028-06-30
Starting the 4 wheeler!!! Engine-on
car max speed is 160 km/h — ride safe!
Registration Number : TN36 CC 1111
```

```
Color : Black
Maximum Speed in km/hr : 160
Number of Wheels : 4
Insurance Expiry date in YYYY-MM-DD : 2029-06-30
The Car has sun roof
```

```
car2 = Car('White', 180, 'TN36 CC 2222', date(2027, 6, 30), False)

car2.get_registration_number()
print(car2.get_insurance_expiry_date())
car2.update_insurance(date(2028, 6, 30))
car2.start_vehicle()
car2.speed_limit_alert()
car2.show()
```

```
2027-06-30
Starting the 4 wheeler!!! Engine-on
car max speed is 180 km/h — ride safe!
Registration Number : TN36 CC 2222
Color : White
Maximum Speed in km/hr : 180
Number of Wheels : 4
Insurance Expiry date in YYYY-MM-DD : 2028-06-30
The Car has no sun roof
```

The following code explains how private and protected data can be modified with highly irrelevant data potentially causing danger, when used data mangling

```
# name mangling for Private variables

print("auto1 secret details direct access")
auto1._Vehicle__registration_number = 'TN36 AA 9999'
auto1._Vehicle__insurance_expiry_date = date(2127, 12, 31)
print(auto1._Vehicle__registration_number)
print(auto1._Vehicle__insurance_expiry_date)
print("-"*20)

print("bike1 secret details direct access")
bike1._Vehicle__registration_number = 'TN36 BB 9999'
bike1._Vehicle__insurance_expiry_date = date(2128, 12, 31)
print(bike1._Vehicle__registration_number)
print(bike1._Vehicle__insurance_expiry_date)
print("-"*20)

print("car1 secret details direct access")
car1._Vehicle__registration_number = 'TN36 CC 9999'
car1._Vehicle__insurance_expiry_date = date(2129, 6, 30)
print(car1._Vehicle__registration_number)
print(car1._Vehicle__insurance_expiry_date)
print("-"*20)

## directly accessing protected variable

print("auto1 protected details direct access")
auto1._max_speed = 1000
auto1._num_of_wheels = 30
print(auto1._max_speed)
print(auto1._num_of_wheels)
print("-"*20)

print("bike1 protected details direct access")
bike1._max_speed = 2000
bike1._num_of_wheels = 20
print(bike1._max_speed)
print(bike1._num_of_wheels)
print("-"*20)

print("car1 protected details direct access")
car1._max_speed = 3000
car1._num_of_wheels = 40
print(car1._max_speed)
print(car1._num_of_wheels)
```

```
auto1 secret details direct access
TN36 AA 9999
2127-12-31
```

```
--------------------
bike1 secret details direct access
TN36 BB 9999
2128-12-31
--------------------
car1 secret details direct access
TN36 CC 9999
2129-06-30
--------------------
auto1 protected details direct access
1000
30
--------------------
bike1 protected details direct access
2000
20
--------------------
car1 protected details direct access
3000
40
```

Question 8

Q8. Given a list of 50 natural numbers from 1-50. Create a function that will take every element from the list and return the square of each element. Use the python map and filter methods to implement the function on the given list.

```
natural_nums = [num for num in range(1,51)]


print(natural_nums)
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,
```

```
def square(num):
  return num**2


even_sq_of_natural_nums = list(filter(lambda sq_num: sq_num % 2 ==0 , map(lambda num: num ** 2, natural_nums)))


print(even_sq_of_natural_nums)
```

```
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400, 484, 576, 676, 784, 900, 1024, 1156, 1296, 1444, 1600, 1764, 1936, 2116, 2304, 2500]
```

```
sq_of_even_natural_nums = list(map(lambda even_num: even_num ** 2, filter(lambda num:num % 2 == 0, natural_nums)))


print(sq_of_even_natural_nums)
```

```
[4, 16, 36, 64, 100, 144, 196, 256, 324, 400, 484, 576, 676, 784, 900, 1024, 1156, 1296, 1444, 1600, 1764, 1936, 2116, 2304, 2500]
```

```
# since both look equal because a square of a even natural number is always even
# so, we shall try square of (natural numbers divisible by 3)
sq_of_multiples_of_3 = list(map(lambda even_num: even_num ** 2, filter(lambda num:num % 3 == 0, natural_nums)))


print(sq_of_multiples_of_3)
```

```
[9, 36, 81, 144, 225, 324, 441, 576, 729, 900, 1089, 1296, 1521, 1764, 2025, 2304]
```

Question 9

Q9. Create a class, Triangle. Its init() method should take self, angle1, angle2, and angle3 as arguments.

Question 10

Q10. Create a class variable named number_of_sides and set it equal to 3.

Question 11

Q11. Create a method named check_angles. The sum of a triangle's three angles should return True if the sum is equal to 180, and False otherwise. The method should print whether the angles belong to a triangle or not.

Question 11.1

11.1 Write methods to verify if the triangle is an acute triangle or obtuse triangle.

```
class Triangle:
  number_of_sides = 3

  def __init__(self, angle1, angle2, angle3):
    self.angle1 = angle1
    self.angle2 = angle2
    self.angle3 = angle3

  def __sum_of_angles(self):
    return self.angle1 + self.angle2 + self.angle3

  def __is_sum_of_angles_180(self):
    return True if self.__sum_of_angles() == 180 else False

  def check_angles(self):
    if(self.__is_sum_of_angles_180()):
      print(f"The given angles {self.angle1}, {self.angle2} and {self.angle3} belong to a triangle")
    else:
      print(f"The given angles {self.angle1}, {self.angle2} and {self.angle3} do NOT belong to a triangle")
    return self.__is_sum_of_angles_180()

  def is_acute(self):
    if self.__is_sum_of_angles_180() and self.angle1 < 90 and self.angle2 < 90 and self.angle3 < 90:
      return True
    else:
      return False

  def is_obtuse(self):
    if self.__is_sum_of_angles_180() and (self.angle1 > 90 or self.angle2 > 90 or self.angle3 > 90):
      return True
    else:
      return False

  def show_triangle_type(self):
    if self.check_angles():
      if self.is_acute():
        print("it is an acute triangle")
      elif self.is_obtuse():
        print("it is an obtuse triangle")
      else:
        print("Neither acute nor obtuse, it is a right triangle")
```

Question 11.2

11.2 Create an instance of the triangle class and call all the defined methods.

1. Acute Triangle instance

```
acute_triangle1 = Triangle(50,80,50)
acute_triangle1.check_angles()
```

```
The given angles 50, 80 and 50 belong to a triangle
True
```

```
print(acute_triangle1.is_acute())
print(acute_triangle1.is_obtuse())
acute_triangle1.show_triangle_type()
```

```
True
False
The given angles 50, 80 and 50 belong to a triangle
it is an acute triangle
```

2. Acute Triangle instance

```
obtuse_triangle1 = Triangle(20,120,40)
obtuse_triangle1.check_angles()
```

```
The given angles 20, 120 and 40 belong to a triangle
True
```

```
print(obtuse_triangle1.is_acute())
print(obtuse_triangle1.is_obtuse())
obtuse_triangle1.show_triangle_type()
```

```
False
True
The given angles 20, 120 and 40 belong to a triangle
it is an obtuse triangle
```

Question 11.3

11.3 Create three child classes of triangle class - isosceles_triangle, right_triangle and equilateral_triangle.

Question 11.4

11.4 Define methods which check for their properties

3. Neither acute nor obtuse, instance for right triangle

```
right_triangle1 = Triangle(35,90,55)
right_triangle1.check_angles()
```

```
The given angles 35, 90 and 55 belong to a triangle
True
```

```
print(right_triangle1.is_acute())
print(right_triangle1.is_obtuse())
right_triangle1.show_triangle_type()
```

```
False
False
The given angles 35, 90 and 55 belong to a triangle
Neither acute nor obtuse, it is a right triangle
```

4. Instance for not a triangle

```
not_a_triangle1 = Triangle(35,100,55)
not_a_triangle1.check_angles()
```

```
The given angles 35, 100 and 55 do NOT belong to a triangle
False
```

```
print(not_a_triangle1.is_acute())
print(not_a_triangle1.is_obtuse())
not_a_triangle1.show_triangle_type()
```

```
False
False
The given angles 35, 100 and 55 do NOT belong to a triangle
```

```
class IsoscelesTriangle(Triangle):
  def __init__(self, angle1, angle2, angle3):
    super().__init__(angle1, angle2, angle3)

  def check_angles(self):
    if super().check_angles():
      if self.angle1 == self.angle2 or self.angle2 == self.angle3 or self.angle1 == self.angle3:
        print("it is an isosceles triangle")
        return True
      else:
        print("it is NOT an isocesles triangle")
    return False
```

```
class RightTriangle(Triangle):
  def __init__(self, angle1, angle2, angle3):
    super().__init__(angle1, angle2, angle3)

  def check_angles(self):
    if super().check_angles():
      if self.angle1 == 90 or self.angle2 == 90 or self.angle3 == 90:
        print("it is a right triangle")
        return True
      else:
        print("it is NOT a right triangle")
    return False

class EquilateralTriangle(Triangle):
  def __init__(self, angle1, angle2, angle3):
    super().__init__(angle1, angle2, angle3)

  def check_angles(self):
    if super().check_angles():
      if self.angle1 == self.angle2 == self.angle3 == 60:
        print("it is an equilateral triangle")
        return True
      else:
        print("it is NOT an equilateral triangle")
    return False


20 == 20 == 10 ==10
```

```
False
```

Question 12

Q12. Create a class isosceles_right_triangle which inherits from isosceles_triangle and right_triangle.

Question 12.1

12.1 Define methods which check for their properties.

```
class IsoscelesRightTriangle(IsoscelesTriangle, RightTriangle):
  def __init__(self, angle1, angle2, angle3):
    super().__init__(angle1, angle2, angle3)

  def check_angles(self):
    if IsoscelesTriangle.check_angles(self) and RightTriangle.check_angles(self):
      print("it is an isosceles right triangle")
      return True
    else:
      print("it is NOT an isosceles right triangle")
    return False


i = IsoscelesRightTriangle(45,90,45)


i.check_angles()
```

```
The given angles 45, 90 and 45 belong to a triangle
it is a right triangle
it is an isosceles triangle
The given angles 45, 90 and 45 belong to a triangle
it is a right triangle
it is an isosceles right triangle
True
```